Week 2: EDA and Data Preprocessing

DATASET OVERVIEW

1. Importing the necessary libraries

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

2. Importing the dataset

In [2]:
```
df = pd.read_csv("student-por.csv", sep=';', encoding='utf-8')
df.head(5)
```

Out[2]:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	..
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	.
1	GP	F	17	U	GT3	T	1	1	at_home	other	.
2	GP	F	15	U	LE3	T	1	1	at_home	other	.
3	GP	F	15	U	GT3	T	4	2	health	services	.
4	GP	F	16	U	GT3	T	3	3	other	other	.

5 rows × 33 columns

3. The dataset shape (rows and columns)

In [3]:
```
df.shape
```

Out[3]:
```
(649, 33)
```

4. Column types

In [4]:
```
df.dtypes
```

Out[4]:
```
school        object
sex           object
age            int64
address       object
famsize       object
Pstatus       object
Medu           int64
Fedu           int64
Mjob          object
Fjob          object
reason        object
guardian      object
traveltime     int64
studytime      int64
failures       int64
schoolsup     object
famsup        object
paid          object
activities    object
nursery       object
higher        object
internet      object
romantic      object
famrel         int64
freetime       int64
goout          int64
Dalc           int64
Walc           int64
health         int64
absences       int64
G1             int64
G2             int64
G3             int64
dtype: object
```

Notes: There are 17 categorical columns that need to be encoded.

5. Checking for missing values in each column

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
school        0
sex           0
age           0
address       0
famsize       0
Pstatus       0
Medu          0
Fedu          0
Mjob          0
Fjob          0
reason        0
guardian      0
traveltime    0
studytime     0
failures      0
schoolsup     0
famsup        0
paid          0
activities    0
nursery       0
higher        0
internet      0
romantic      0
famrel        0
freetime      0
goout         0
Dalc          0
Walc          0
health        0
absences      0
G1            0
G2            0
G3            0
dtype: int64
```

Notes: There are no missing values. So we don't need to fill any columns.

6. Showing basic statistics for numerical columns

In [6]:
```
df.describe()
```

Out[6]:

	age	Medu	Fedu	traveltime	studytime	failures
count	649.000000	649.000000	649.000000	649.000000	649.000000	649.000000
mean	16.744222	2.514638	2.306626	1.568567	1.930663	0.221880
std	1.218138	1.134552	1.099931	0.748660	0.829510	0.593235
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000
25%	16.000000	2.000000	1.000000	1.000000	1.000000	0.000000
50%	17.000000	2.000000	2.000000	1.000000	2.000000	0.000000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000

7. Showing basic statistics for categorical columns

In [7]:
```
df.describe(include = 'O')
```

Out[7]:

	school	sex	address	famsize	Pstatus	Mjob	Fjob	reason	guardian	sc
count	649	649	649	649	649	649	649	649	649	
unique	2	2	2	2	2	5	5	4	3	
top	GP	F	U	GT3	T	other	other	course	mother	
freq	423	383	452	457	569	258	367	285	455	

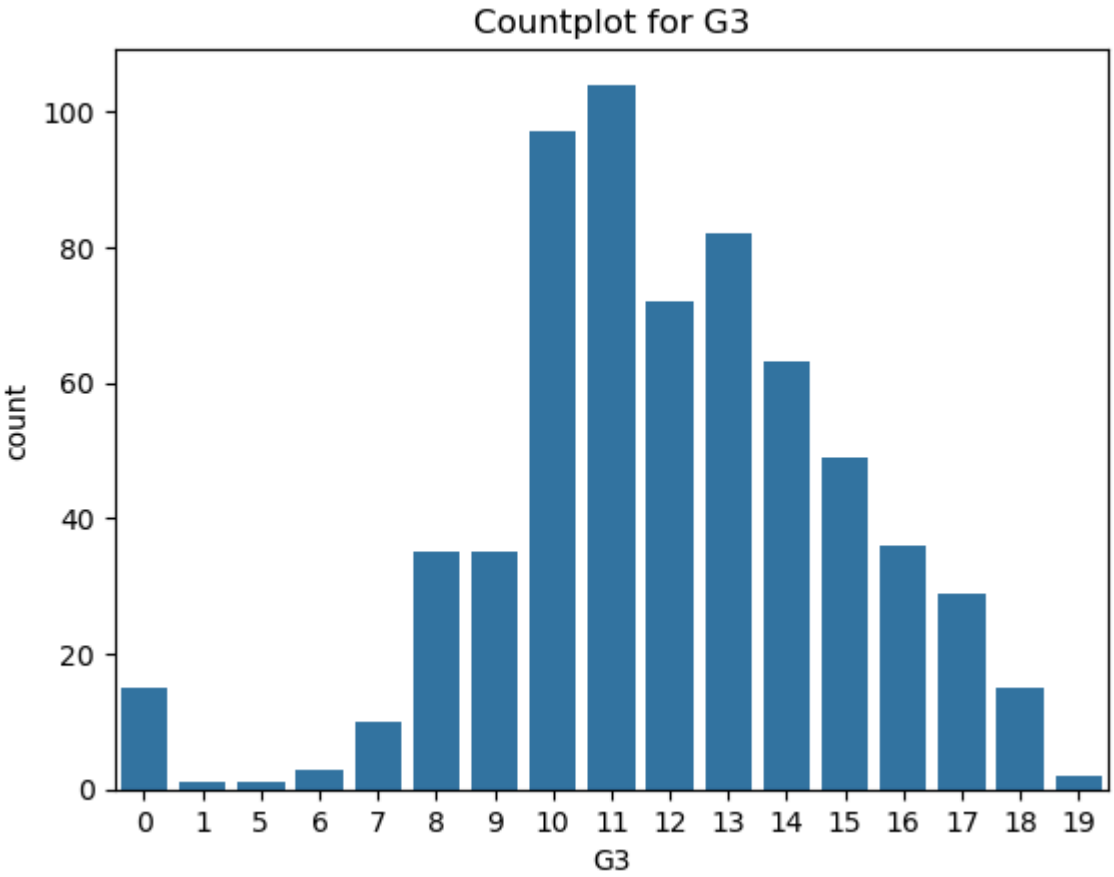
Summary

The dataset contains 30 features.
There are 17 categorical columns that need to be encoded, while the remaining features are numeric or ordinal.
No missing values were found, simplifying preprocessing.
Our target variable is G3, the students' final grade, which we aim to predict based on the other features.

VISUALISATIONS

1. Countplot for target variable G3

In [8]:
```
sns.countplot(data = df, x = 'G3')
plt.title('Countplot for G3')
plt.show()
```

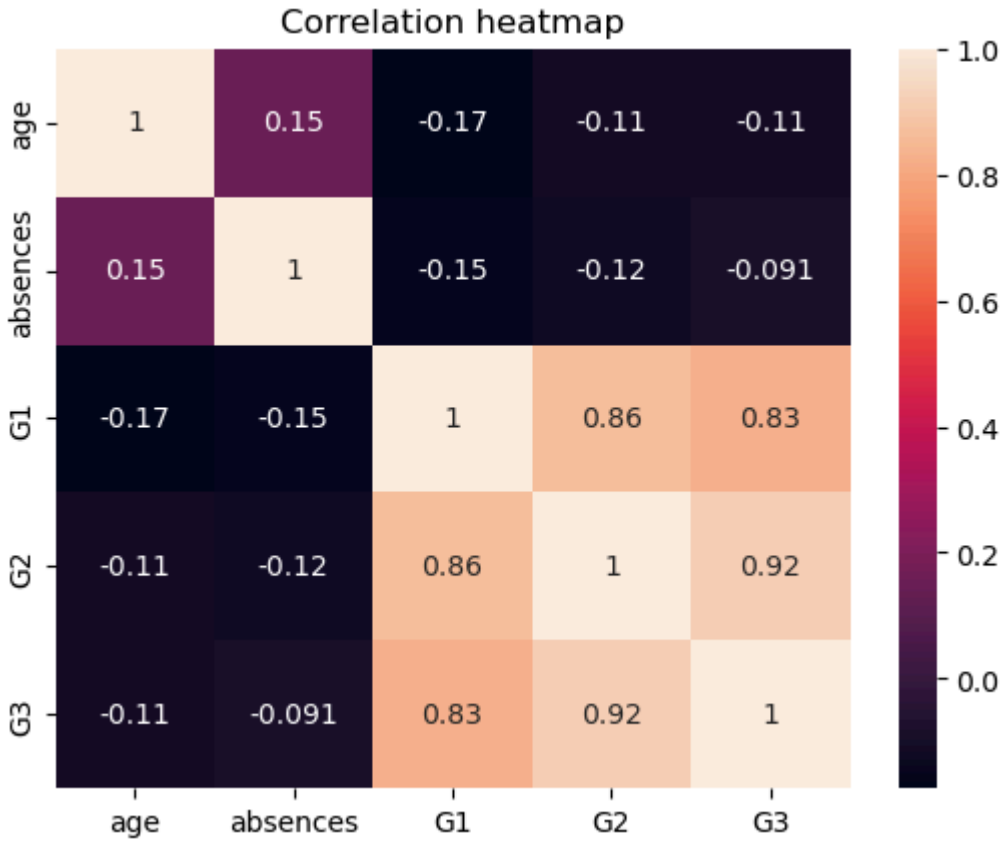


The graph shows the distribution of the target variable G3. We see that most students get grades in the range of 10-11 points. There is also a noticeable peak around 0, which may indicate students who did not take the final exam or were not admitted. This is important to consider, as these "outliers" can affect the accuracy of the model

2. Correlation Heatmap: Correlation matrix.

In [9]:
```
correlation = ['age', 'absences', 'G1', 'G2', 'G3']
corr_matrix = df[correlation].corr()
```

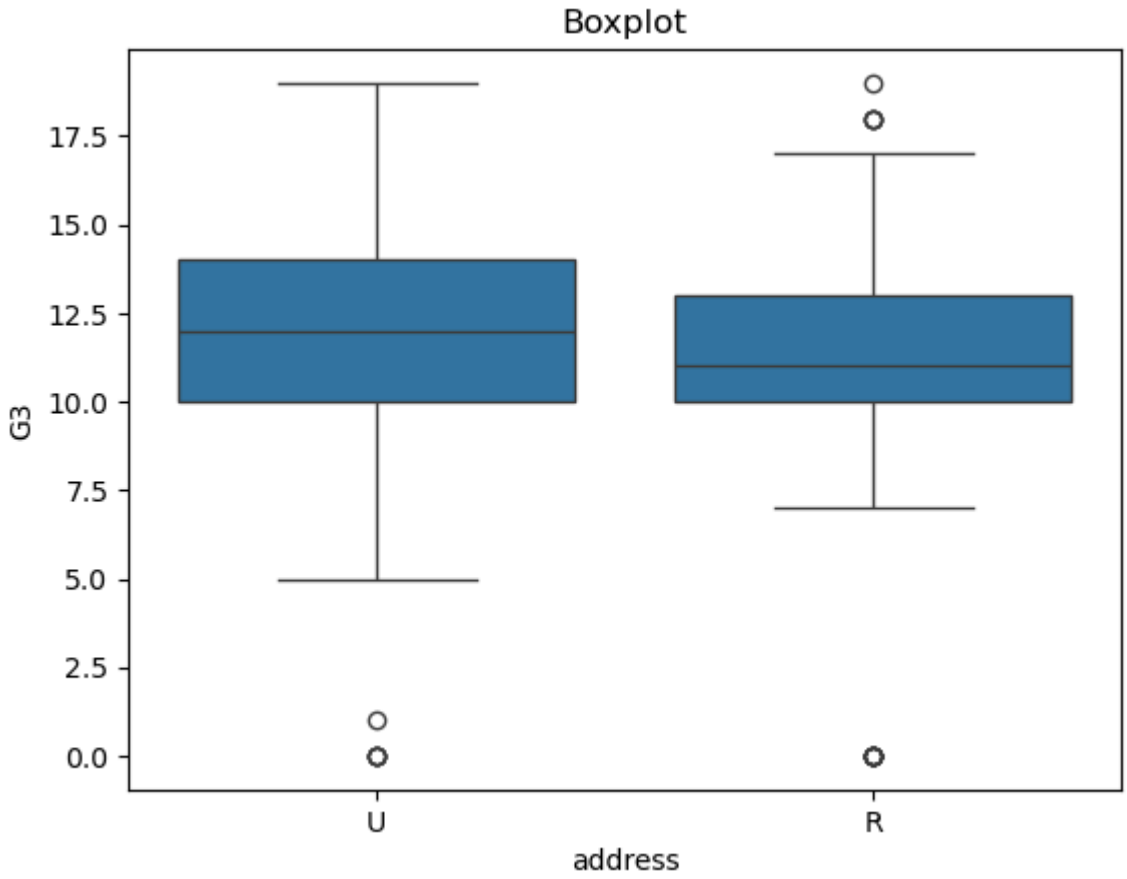
In [10]:
```
sns.heatmap(corr_matrix, annot=True)
plt.title('Correlation heatmap')
plt.show()
```



The heatmap revealed a strong positive correlation between G1, G2 and the final G3 score (coefficient > 0.8). This is logical, since intermediate successes directly affect the result. There is also a weak negative correlation between the number of absences and grades, which supports the hypothesis that attendance is important for academic performance.

3. Boxplot: The impact of residence on grades

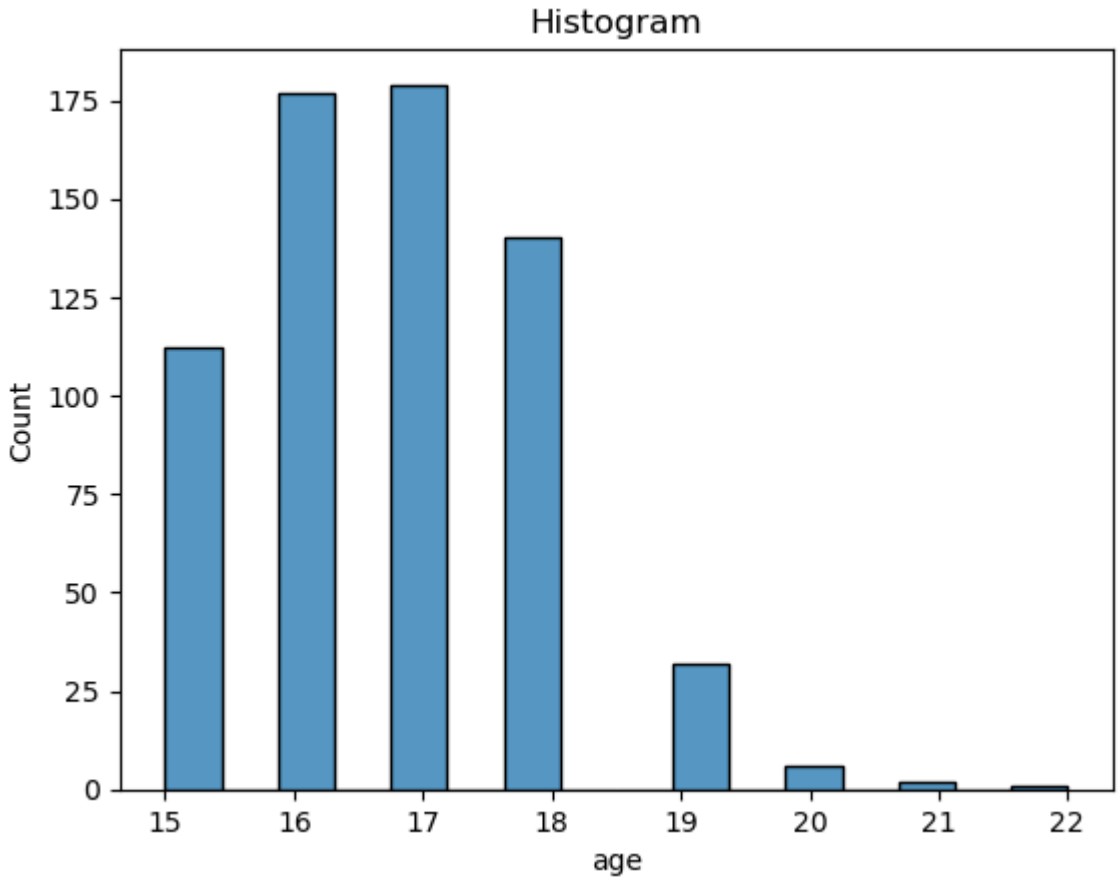
In [11]:
```
sns.boxplot(x = 'address', y = 'G3', data = df)
plt.title('Boxplot')
plt.show()
```



A comparison of the median grades of students from urban (U) for 0 and rural (R) for 1 areas shows that urban students have slightly higher scores on average. The spread (interquartile range) among urban students is also wider, which indicates a greater variety of training levels in an urban environment compared to rural ones.

4. Histogram: Age distribution of students

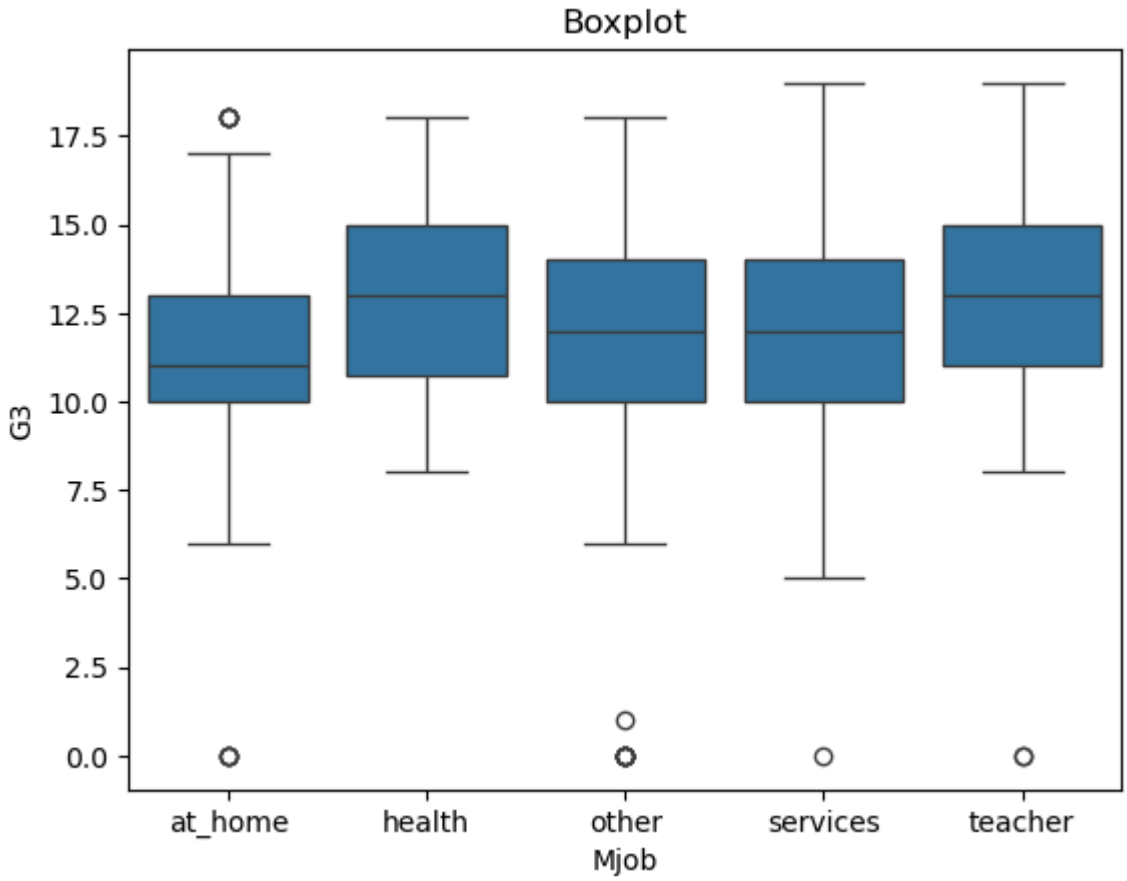
In [12]:
```
sns.histplot(data = df, x = 'age')
plt.title('Histogram')
plt.show()
```



The age histogram shows that the majority of students are aged 15-18 years. The number of students over the age of 19 is sharply decreasing. This means that our model will be better trained to predict outcomes for the typical school age, and for rare categories (20-22 years old), the data may not be sufficient to accurately generalize.

5. Boxplot: Mother's Job and Academic performance

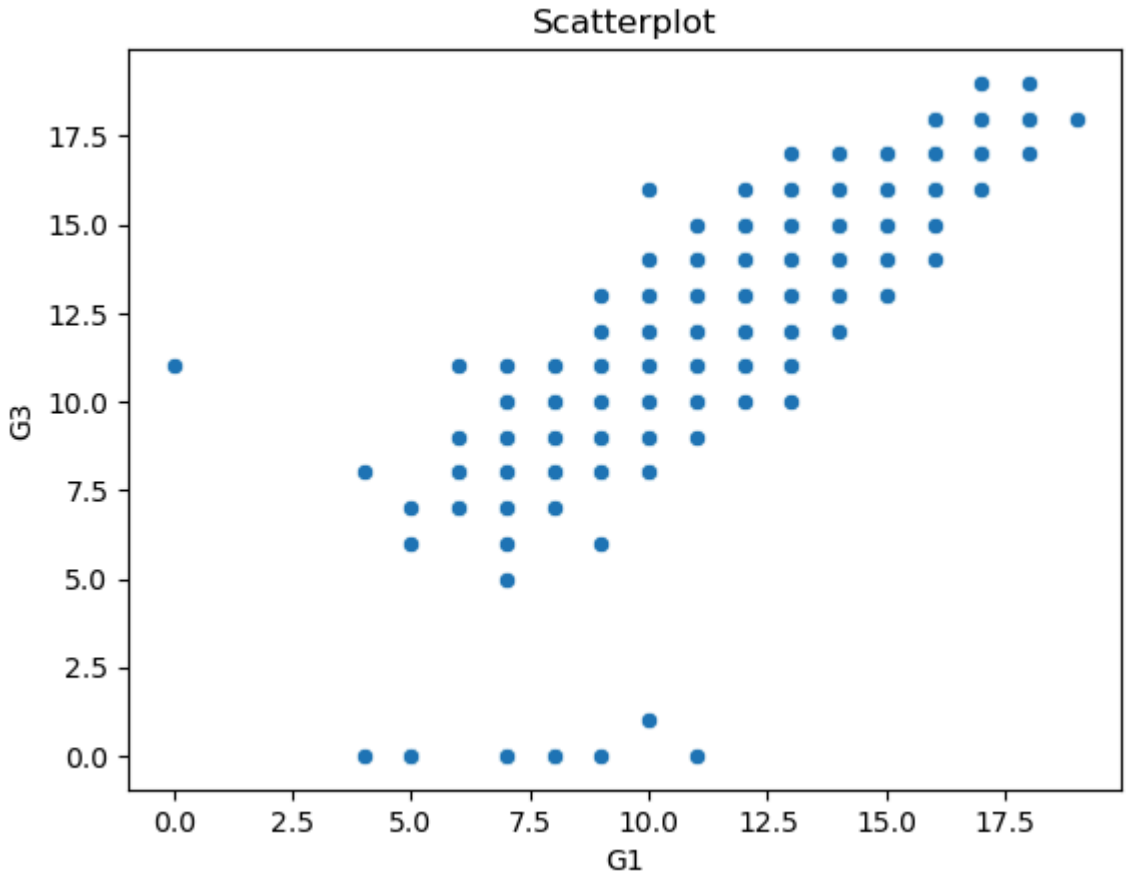
In [13]:
```
sns.boxplot(data = df, x = 'Mjob', y = 'G3')
plt.title('Boxplot')
plt.show()
```



An analysis of the scale graphs shows that the children of mothers working in the field of healthcare (health) or education (teacher), on average, show higher median scores. This confirms that the social status and employment of parents are significant features for predicting academic success.

6. Scatter Plot: Relationship between first grade and final grade

In [14]:
```
sns.scatterplot(x = 'G1', y = 'G3', data = df)
plt.title('Scatterplot')
plt.show()
```



The scatterplot confirms the linear relationship between the results of the first period (G1) and the outcome (G3). Students who get a high score at the start rarely fail the finals. For the machine learning model, the G1 attribute is likely to become one of the most significant factors in decision-making.

MISSING VALUES & FEATURE ENGINEERING

1. Encoding categorical columns

In [15]:
```
#binary encoding

df['sex'] = df['sex'].map({'F': 0, 'M': 1})                #Female - 0, Ma
df['school'] = df['school'].map({'GP': 0, 'MS': 1})        #Gabriel Pereir
df['address'] = df['address'].map({'U': 0, 'R': 1})        #urban - 0, rur
df['famsize'] = df['famsize'].map({'LE3': 0, 'GT3':1})     #less than 3 -
df['Pstatus'] = df['Pstatus'].map({'T': 0, 'A': 1})        #living togethe
df['schoolsup'] = df['schoolsup'].map({'yes': 1, 'no': 0})
df['famsup'] = df['famsup'].map({'yes': 1, 'no': 0})
df['paid'] = df['paid'].map({'yes': 1, 'no': 0})
df['activities'] = df['activities'].map({'yes': 1, 'no': 0})
df['nursery'] = df['nursery'].map({'yes': 1, 'no': 0})
df['higher'] = df['higher'].map({'yes': 1, 'no': 0})
df['internet'] = df['internet'].map({'yes': 1, 'no': 0})
df['romantic'] = df['romantic'].map({'yes': 1, 'no': 0})
```

Notes: Binary categorical variables were encoded as 0/1, where 1 indicates the presence of a feature.

In [16]:
```
# one-hot encoding

df = pd.get_dummies(df, columns=['Mjob', 'Fjob', 'reason', 'guardian'], d
```

Notes: Nominal categorical variables without an inherent order (Mjob, Fjob, reason, guardian) were transformed using one-hot encoding.

2. Creating new columns

In [17]:
```
df['G_avg'] = (df['G1'] + df['G2']) / 2
df['G_improvement'] = (df['G2'] > df['G1']).astype(int)
```

Notes: Creating a new G_avg and G_improvement attribute that combines the intermediate estimates. This helps the model capture the overall dynamics of student performance before the final exam.

In [18]:
```
#checking the column types after the encoding

df.dtypes
```

Out[18]:
```
school                 int64
sex                    int64
age                    int64
address                int64
famsize                int64
Pstatus                int64
Medu                   int64
Fedu                   int64
traveltime             int64
studytime              int64
failures               int64
schoolsup              int64
famsup                 int64
paid                   int64
activities             int64
nursery                int64
higher                 int64
internet               int64
romantic               int64
famrel                 int64
freetime               int64
goout                  int64
Dalc                   int64
Walc                   int64
health                 int64
absences               int64
G1                     int64
G2                     int64
G3                     int64
Mjob_health            int64
Mjob_other             int64
Mjob_services          int64
Mjob_teacher           int64
Fjob_health            int64
Fjob_other             int64
Fjob_services          int64
Fjob_teacher           int64
reason_home            int64
reason_other           int64
reason_reputation      int64
guardian_mother        int64
guardian_other         int64
G_avg                float64
G_improvement          int64
dtype: object
```

In [19]:
```
# displaying the table after encoding

df.head(5)
```

Out[19]:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	traveltime	studytim
0	0	0	18	0	1	1	4	4	2	
1	0	0	17	0	1	0	1	1	1	
2	0	0	15	0	0	0	1	1	1	
3	0	0	15	0	1	0	4	2	1	
4	0	0	16	0	1	0	3	3	1	

5 rows × 44 columns

Summary

Binary categorical features were mapped to 0/1.
Nominal categorical features without a natural order were converted into separate columns for each category using one-hot encoding to avoid introducing artificial ordinality.
Ordinal and numerical features were kept as integers.
These preprocessing steps ensure the dataset is ready for modeling.

PREPROCESSING PIPELINE & SPLIT

Scaling & Splitting

In [20]:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
#importing necessery libraries for train_test_split

X = df[['studytime', 'absences', 'health', 'G1', 'G2', 'G_avg', 'G_improv
y = df['G3'] #target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

numeric_features = ['studytime', 'absences', 'health', 'G1', 'G2', 'G_avg

scaler = StandardScaler() #initialization and declaring with variable

X_train[numeric_features] = scaler.fit_transform(X_train[numeric_features
X_test[numeric_features] = scaler.transform(X_test[numeric_features]) #tr

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
# Data was correctly split
```

```
(519, 7)
(130, 7)
(519,)
(130,)
```

This is necessary so that features with large values (for example, the number of gaps) do not overwhelm features with small values (for example, estimates) when calculating model weights.